Dataset Notes: C-MAPSS — Deep Reference

Saxena, Goebel, Simon, Eklund — PHM'08 dataset — *Column-by-column edition*

Contents

Overview: Four Sub-Datasets

Figure 1 shows the four C-MAPSS sub-datasets and their key properties. All share the same 26-column file format.

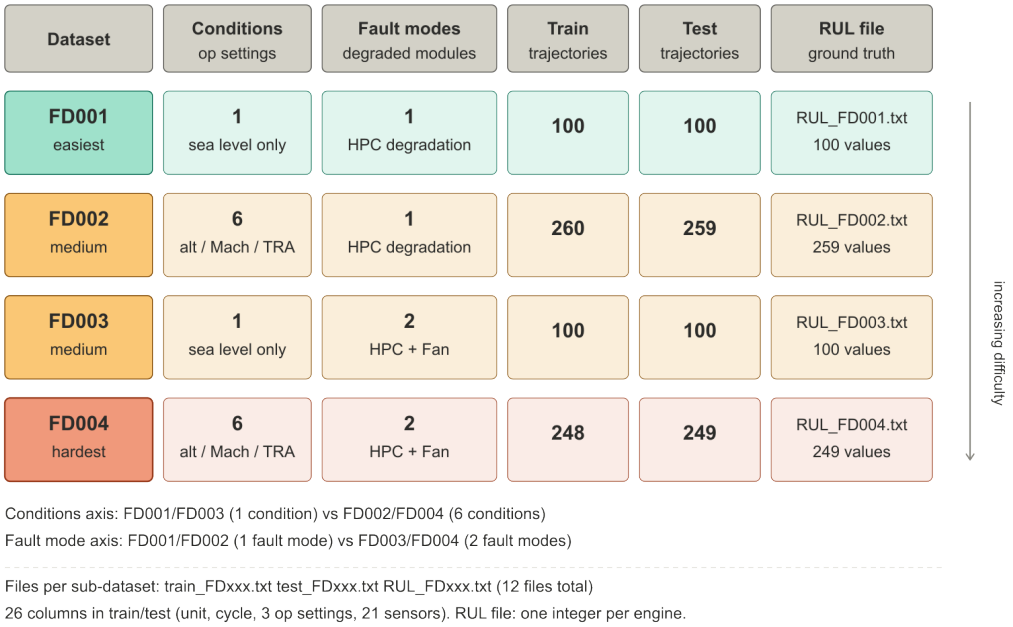
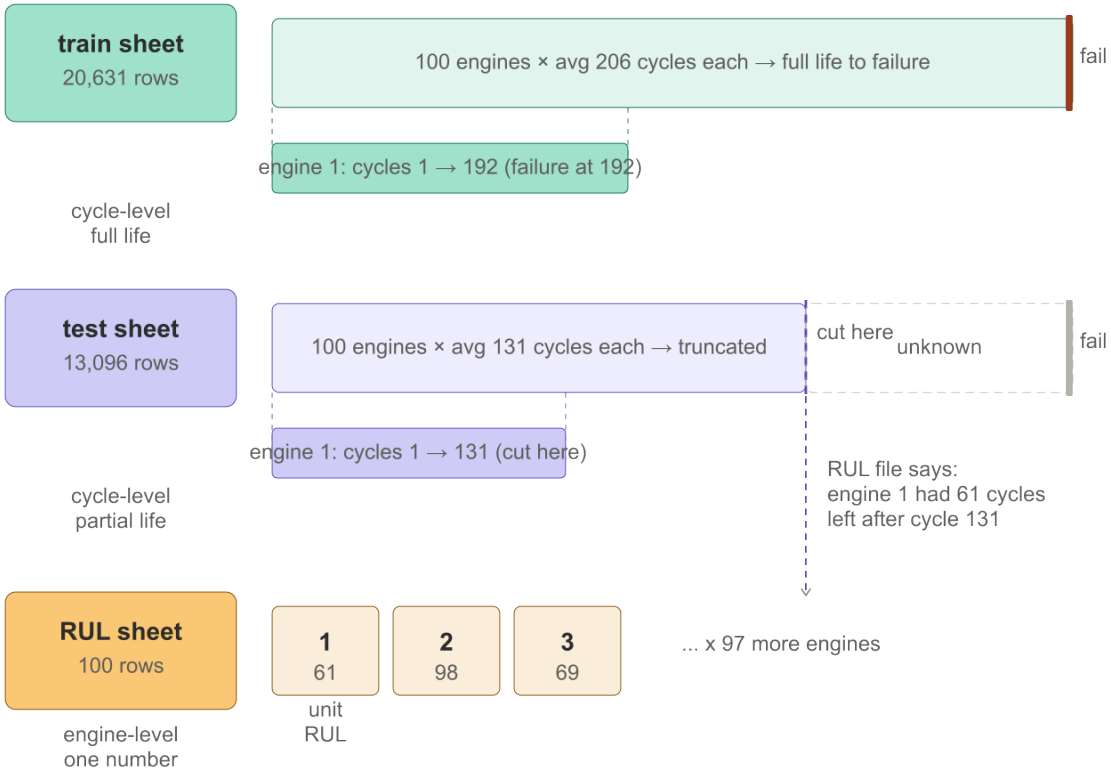


Figure 1: The four C-MAPSS sub-datasets. FD001 is simplest (1 condition, 1 fault mode). FD004 is hardest (6 conditions, 2 fault modes).

The Three Files and Their Different Lengths

Why the lengths differ

Each sub-dataset produces three files. They differ in length because they represent the same engines at different levels of granularity.



sheet	granularity	row count	why different
train	one row / cycle	20,631	full lives, all cycles
test	one row / cycle	13,096	truncated lives
RUL	one row / engine	100	one number per engine

Figure 2: The three sheets for FD001. Train and test are cycle-level (many rows per engine). RUL is engine-level (one row per engine). The bottom table summarises the three granularities.

File	Granularity	FD001 rows	Why
`train_FD001.txt`	one row per cycle	20,631	100 engines × avg 206 cycles to failure
`test_FD001.txt`	one row per cycle	13,096	100 engines × avg 131 cycles (truncated)
`RUL_FD001.txt`	one row per engine	100	one RUL value per test engine

How the three files connect

The link between all three files is `unit_number`. It is the key for any join operation.

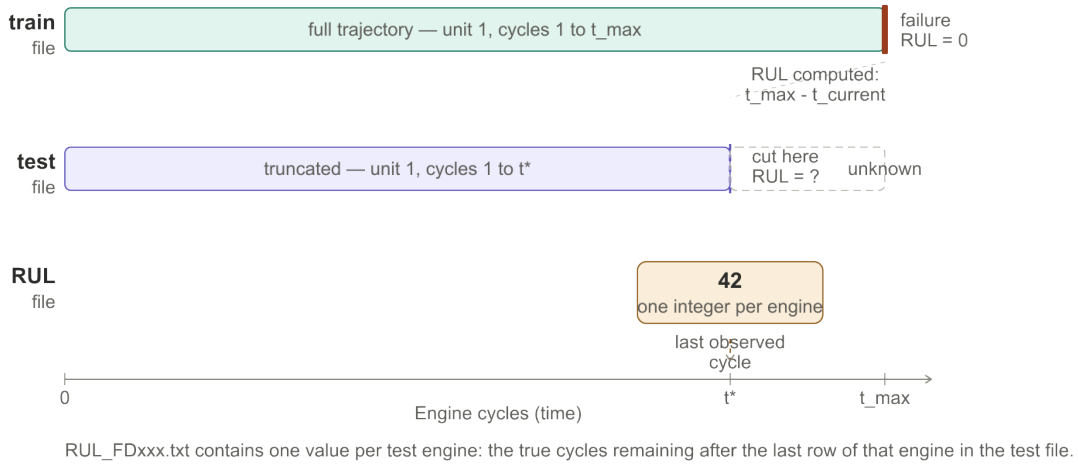


Figure 3: The relationship between the three files for one engine. The train file runs to failure. The test file stops at t^*. The RUL file gives one number per engine: the true cycles remaining at t^*.

The three files are three zoom levels of the same engines:

`train`	cycle-level	full life, all sensors, all cycles
`test`	cycle-level	partial life, sensors only
`RUL`	engine-level	one number: true RUL at cut point

They are connected by `unit_number`. Analysis that combines files must always join on this column.

What this means for analysis

Different questions require different combinations:

Question	Files used	How
What does degradation look like?	train only	Plot sensor vs cycle for each engine
How well did I predict RUL?	test + RUL, join on unit_number	Compare predicted RUL to true RUL
What is the engine lifespan distribution?	RUL + last_cycle (v3)	total_life_est = last_cycle + RUL
Do long-lived engines degrade differently?	train + RUL, join on unit_number	Group train rows by engine life length

Input Columns: All 26 Raw Columns

Every `train` and `test` row has 26 columns. They are grouped below by physical meaning. Figure 4 shows where each group lives on the engine.

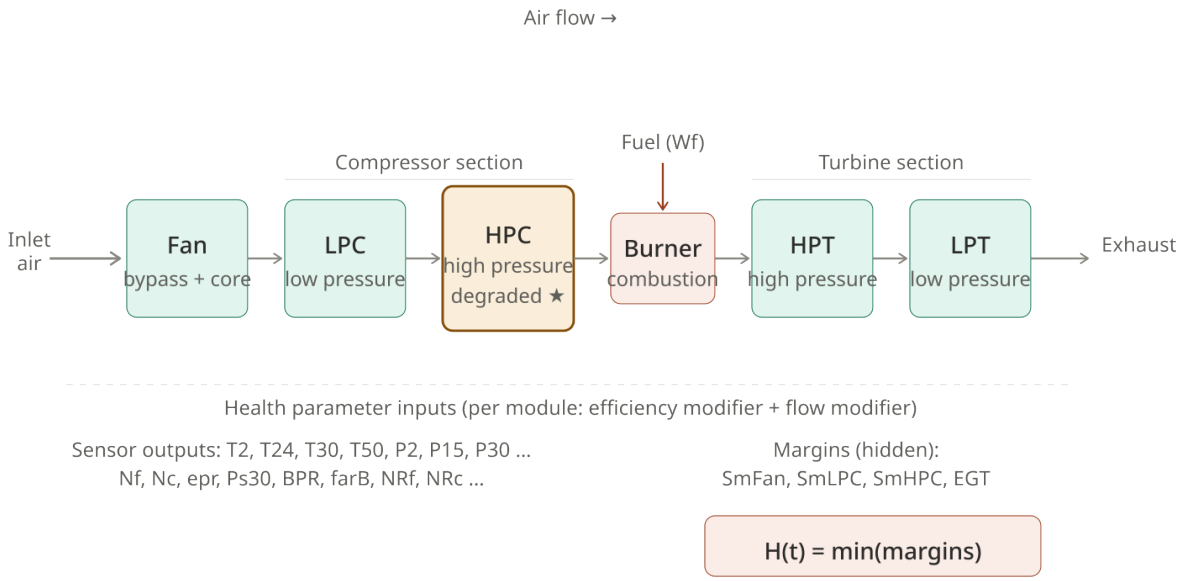


Figure 4: C-MAPSS turbofan engine. Sensors measure temperatures and pressures at inlet, inter-stage, and outlet locations. The amber HPC module is the one degraded in FD001/FD002.

Table 1: All 26 raw input columns grouped by physical meaning.

Col	Name	Units	Physical meaning
Group A — Identifiers			
1	`unit_number`	—	Engine ID. Integer. Resets per file. Every analysis groups or filters on this.
2	`time_cycles`	cycles	Cycle counter per engine. Starts at 1. One cycle ≈ one flight.
Group B — Operating conditions			
3	`op_setting_1`	ft	Altitude. Controls ambient pressure and temperature. Ranges 0–42,000 ft across 6 conditions.
4	`op_setting_2`	—	Mach number. Flight speed as fraction of speed of sound. Ranges 0–0.84.
5	`op_setting_3`	—	Throttle resolver angle (TRA). Commanded thrust level. Ranges 20–100. Constant within FD001/FD003.
Group C — Temperatures (total = stagnation, measures enthalpy)			
6	`T2`	°R	Total temperature at fan inlet. Reflects ambient conditions. Jumps dramatically across flight conditions.
7	`T24`	°R	Total temperature at LPC outlet / HPC inlet. Rises as air is compressed in LPC.
8	`T30`	°R	Total temperature at HPC outlet / combustor inlet. Key health indicator: rises when HPC degrades.
9	`T50`	°R	Total temperature at LPT outlet. Reflects turbine work extraction and combustor output.
Group D — Pressures			
10	`P2`	psia	Total pressure at fan inlet. Ambient total pressure. Drops at altitude.
11	`P15`	psia	Total pressure in bypass duct (between fan and LPC). Reflects fan performance.
12	`P30`	psia	Total pressure at HPC outlet. Main compression output. Falls as HPC degrades.
16	`Ps30`	psia	Static pressure at HPC outlet (no velocity component). Related to P30 but measured differently.
Group E — Speeds			
13	`Nf`	rpm	Physical fan speed. Actual rotational speed of the fan shaft. Controlled by fan-speed controller.
14	`Nc`	rpm	Physical core speed. Actual rotational speed of the HPC/HPT/combustor shaft.
18	`NRf`	rpm	Corrected fan speed. Adjusted for inlet conditions: $NR_f = N_f/\sqrt{T_2/T_{\text{ref}}}$. Comparable across conditions.
19	`NRc`	rpm	Corrected core speed. Same correction applied to N_c: $NR_c = N_c/\sqrt{T_2/T_{\text{ref}}}$. Comparable across conditions.
Group F — Flow ratios and combustion			
15	`epr`	—	Engine pressure ratio P_{50}/P_2. Overall pressure rise through the engine. Thrust indicator.
17	`phi`	pps/psi	Fuel flow / P_{s30} ratio. Normalised fuel flow. Rises when HPC pressure drops due to degradation.
20	`BPR`	—	Bypass ratio. Mass flow through bypass duct divided by core mass flow. Reflects fan/core balance.
21	`farB`	—	Burner fuel-air ratio. Mass of fuel per mass of air in the combustor. Combustion efficiency indicator.
22	`htBleed`		Bleed enthalpy. Energy extracted from the

Why corrected speeds (NRf, NRc) matter for FD002/FD004. Physical speeds N_f and N_c depend on inlet temperature. At high altitude (cold air), the same thrust requires different RPM than at sea level. Corrected speeds remove this dependency, making the values comparable across flight conditions. For single-condition sub-datasets (FD001/FD003) this distinction is less important.

Which columns carry degradation signal? HPC degradation (FD001/FD002) primarily shows up in: T30 ↑ (outlet temperature rises), P30 ↓ (outlet pressure falls), phi ↑ (more fuel needed to maintain thrust), Ps30 ↓. The other sensors change too, but these four are the most direct.

Additional Input Columns

These three columns are not in the raw files. They are derived from the 26 raw columns and appended in the v3 spreadsheet. They appear on both `train` and `test` sheets.

condition_id

For FD002 and FD004, raw sensor values are not comparable across cycles because the engine operates at different flight conditions. Figure 5 shows this problem clearly.

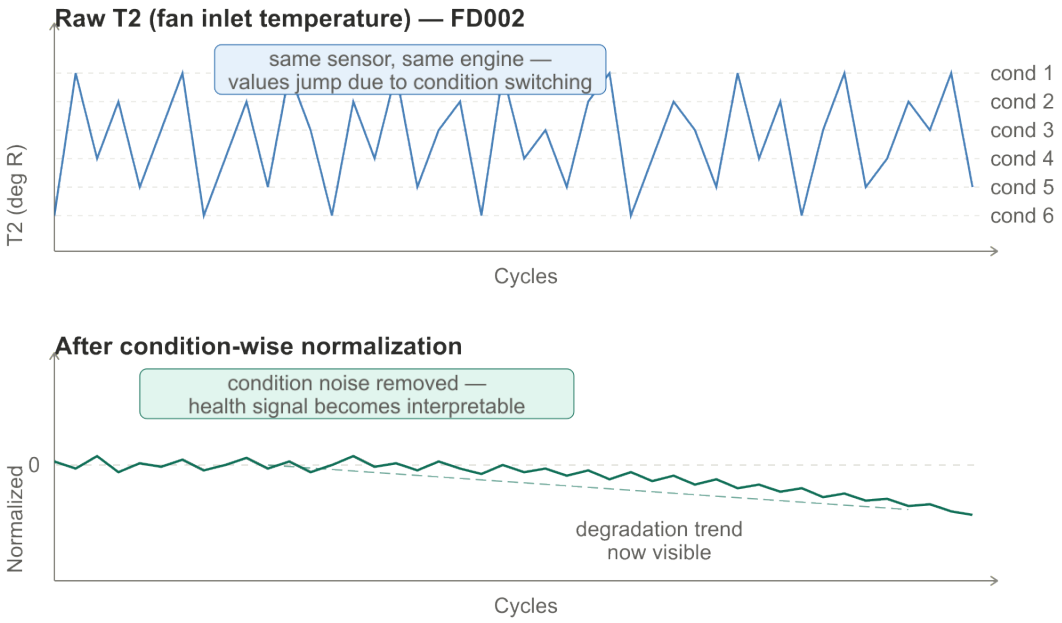


Figure 5: Top: raw T_2 values jumping between six bands as the engine switches flight conditions — not degradation signal. Bottom: after condition-wise normalisation, the health signal becomes visible.

`condition_id` assigns an integer 1–6 to each row identifying which of the six flight conditions that cycle belongs to. It is computed by KMeans clustering on the three operating setting columns, as shown in Figure 6.

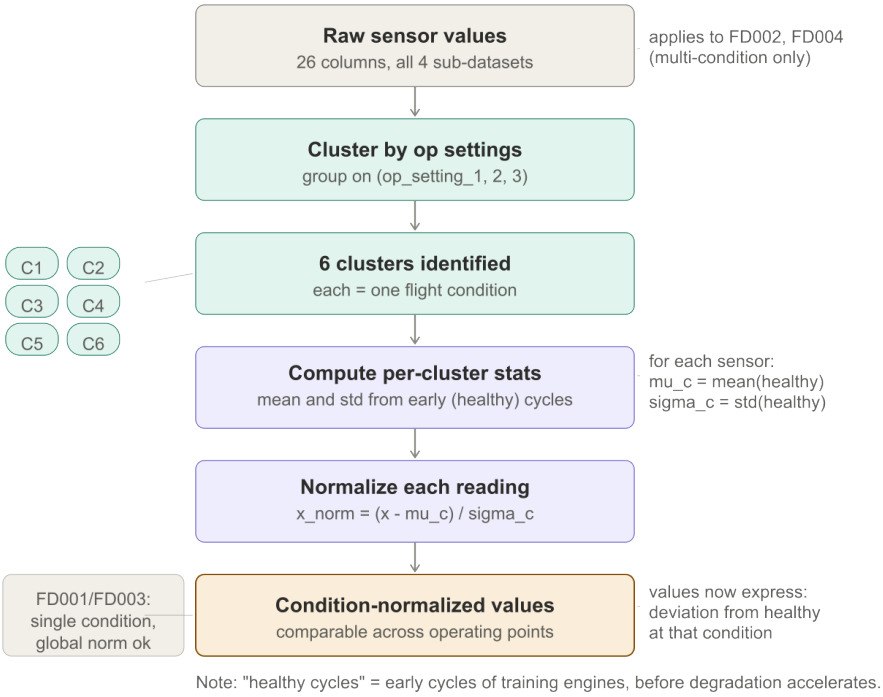


Figure 6: How `condition_id` is produced. KMeans($k = 6$) is fitted on the train set and applied to both train and test. For FD001/FD003, all rows receive `condition_id` = 1.

The KMeans model minimises within-cluster sum of squared distances:

$$J = \sum_{i=1}^{n} \left\| x_i - \mu_{c(i)} \right\|^2 \tag{1}$$

where $x_i = (\texttt{op\_setting\_1}_i, \texttt{op\_setting\_2}_i, \texttt{op\_setting\_3}_i)$, μ_k is the centroid of cluster k, and $c(i) = \arg\min_k \|x_i - \mu_k\|^2$ is the assigned cluster.

> **Important:** `condition_id` labels are not comparable across sub-datasets. Condition 2 in FD002 and condition 2 in FD004 refer to different physical flight points because KMeans assigns labels in arbitrary order. Always compare conditions by their `op_setting` values, not by label number.

dT_compressor

Temperature rise across the HPC:

$$\texttt{dT\_compressor} = T_{30} - T_{24} \tag{2}$$

where T_{30} is the total temperature at the HPC outlet (col 8) and T_{24} is the total temperature at the LPC outlet / HPC inlet (col 7).

Physical meaning: healthy compression raises temperature by a predictable amount. As the HPC degrades, compression efficiency drops. The controller compensates by demanding more fuel, which raises T_{30} further. So `dT_compressor` tends to *increase* as HPC health deteriorates.

Why add this column? The raw T_{30} value is dominated by the operating condition (altitude, throttle). The difference $T_{30} - T_{24}$ removes the common-mode variation and isolates the HPC contribution.

dT_turbine

Temperature drop across the turbine section:

$$\texttt{dT\_turbine} = T_{50} - T_{30} \tag{3}$$

where T_{50} is the total temperature at the LPT outlet (col 9) and T_{30} is the HPC outlet / combustor inlet (col 8).

Physical meaning: the turbines extract work from the hot combustion gases, dropping their temperature. A healthy turbine produces a large drop. As turbine efficiency changes, this delta shifts. Note: in FD001/FD002 the turbines are not the degraded component, so this column is less discriminative there but remains useful as a health indicator for FD003/FD004 where Fan degradation can affect the thermodynamic balance.

Output Columns (Train Sheet Only)

These three columns are the *targets* for machine learning. They appear only on the `train` sheet because the test RUL is the unknown quantity to be predicted.

Figure 7 shows the full derivation map for all additional columns.

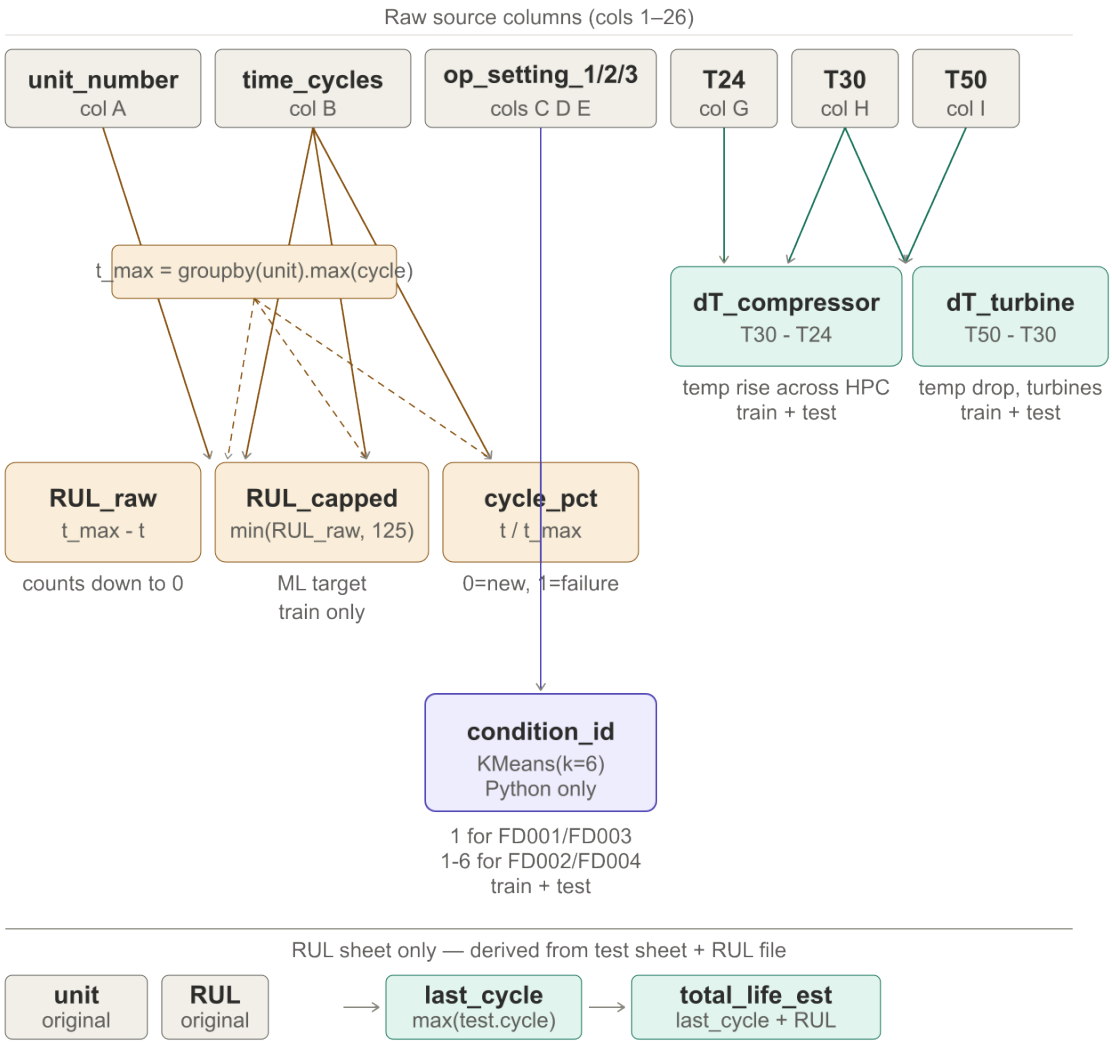


Figure 7: Derivation map for all additional columns. Source columns (gray, top row) feed into three groups of derived columns via different computation paths.

RUL_raw

Remaining useful life in cycles, counting down to zero at failure:

$$\texttt{RUL\_raw}(i) = t_{\max}(u_i) - t_i \tag{4}$$

where:

- t_i is `time_cycles` for row i
- u_i is `unit_number` for row i
- $t_{\max}(u_i) = \max\{t_j : u_j = u_i\}$ is the last cycle observed for that engine (the failure cycle in the training set)

`RUL_raw` $= 0$ at the last row of each training engine and counts upward toward earlier cycles.

Spreadsheet formula (LO Calc):

```
{=MAX(IF($A:$A=$A2,$B:$B))-B2}
Ctrl+Shift+Enter (array formula)
```

RUL_capped

A piece-wise linear target that clamps long-range RUL values:

$$\texttt{RUL\_capped}(i) = \min\big(\texttt{RUL\_raw}(i),\ 125\big) \tag{5}$$

Figure 8 shows the difference between the two shapes.

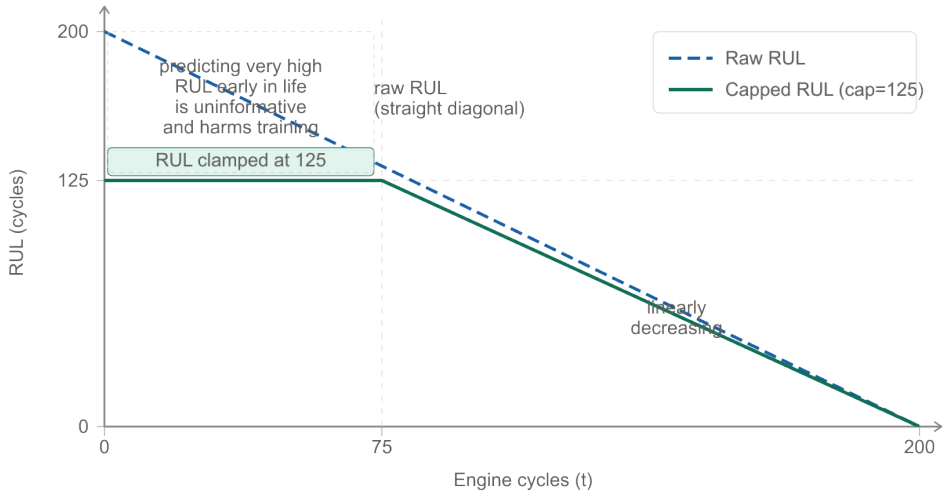


Figure 8: Raw RUL (dashed) vs capped RUL (solid, cap = 125). Early in engine life the signal carries little degradation information; the cap replaces uninformative large values with a flat healthy label.

Why 125? This is a convention in the research literature, not a physical constant. Values from 100 to 150 appear in different papers. The reasoning: sensor signals only start showing degradation trend roughly 100–150 cycles before failure. Asking a model to predict RUL = 300 from a healthy-looking sensor trace is both unreasonable and wastes model capacity.

Spreadsheet formula:

```
=MIN(AD2, 125)
```

(where AD is the RUL_raw column)

cycle_pct

Fractional position in the engine's life, normalised to $[0, 1]$:

$$\texttt{cycle\_pct}(i) = \frac{t_i}{t_{\max}(u_i)} \tag{6}$$

Physical meaning: `cycle_pct` = 0 at the first cycle of an engine and `cycle_pct` = 1 at failure. Unlike `time_cycles`, it is comparable across engines of different total lifespans. Useful for questions like "at 50% of engine life, what do the sensors look like?"

Spreadsheet formula (LO Calc):

```
{=B2/MAX(IF($A:$A=$A2,$B:$B))}
Ctrl+Shift+Enter (array formula)
```

RUL Sheet Columns

The `RUL` sheet has four columns. The first two (`unit_number`, `RUL`) come directly from the NASA `RUL_FDxxx.txt` file. The last two are derived.

last_cycle

The cycle number at which the test trajectory was cut:

$$\texttt{last\_cycle}(u) = \max\{t_j : u_j = u,\ j \in \text{test}\} \tag{7}$$

where u is the engine's `unit_number`.

Where it comes from: this is the maximum `time_cycles` value for that engine in the **test** file. It tells you how far into the engine's life the test trajectory was observed.

Not stated in the paper. This is a logical consequence of the problem definition: the test file contains cycles 1 through t^*, so $t^* =$ `last_cycle`.

Spreadsheet formula (LO Calc):

```
{=MAX(IF(test.A:A=A2,test.B:B))}
Ctrl+Shift+Enter (array formula)
```

total_life_est

Estimated total engine life, combining observed and remaining:

$$\texttt{total\_life\_est}(u) = \texttt{last\_cycle}(u) + \texttt{RUL}(u) \tag{8}$$

Physical meaning: the engine was observed for `last_cycle` cycles in the test set. It had `RUL` cycles remaining. Therefore its estimated total lifespan is their sum.

Not stated in the paper. Derived for interpretability. Allows questions like "what is the distribution of engine lifespans across the test set?"

Spreadsheet formula:

```
=C2+B2
```

(plain formula, no array entry needed)

Equation Reference

Table 2: All derived column equations with variable sources.

Column	Equation	Variables from
`dT_compressor`	$T_{30} - T_{24}$	T30 = col 8, T24 = col 7
`dT_turbine`	$T_{50} - T_{30}$	T50 = col 9, T30 = col 8
`condition_id`	$c(i) = \arg\min_k \|x_i - \mu_k\|^2$	x_i = cols 3,4,5; μ_k from KMeans fit on train
`RUL_raw`	$t_{\max}(u_i) - t_i$	t_i = col 2, $t_{\max}$ = max cycle per engine in train
`RUL_capped`	$\min($`RUL_raw`$, 125)$	RUL_raw (derived above)
`cycle_pct`	$t_i \,/\, t_{\max}(u_i)$	t_i = col 2, $t_{\max}$ = max cycle per engine in train
`last_cycle`	$\max\{t_j : u_j = u\}$ over test	col 2 of test file, filtered by unit_number
`total_life_est`	`last_cycle` + `RUL`	last_cycle (derived), RUL = NASA file

Spreadsheet vs Python boundary. Five of the eight derived columns can be computed as spreadsheet formulas: `dT_compressor`, `dT_turbine`, `RUL_raw`, `RUL_capped`, `cycle_pct`, `last_cycle`, `total_life_est`. Only `condition_id` requires Python, because KMeans is an iterative optimisation algorithm that cannot be expressed as a cell formula.